

Resource Guardrail, Memory Management & CPU Multi-Tenancy Strategy

Executive Technical Summary & System Behavior Breakdown

1. Executive Summary & Root Cause of Initial Warning

Issue Identified: On startup, the application reported 'Only 0.4GB RAM available (3%)' on a machine with 16GB physical RAM.

Root Cause: The LLM startup scripts were using '--mlock' (memory pinning). This forced Windows to lock all ~5.4GB of Gemma-4 model weights, embedding models, and pre-allocated KV-cache permanently into physical RAM, forbidding OS memory page swapping.

Result: Available physical RAM dropped to ~1.5GB at idle. The Resource Guard's initial strict thresholds (12% critical, 1.5GB floor) treated normal idle state with model loaded as an emergency and refused new audits.

Resolution: Removed '--mlock' to allow shared OS memory management, recalibrated Resource Guard thresholds, and removed artificial application-level concurrency queues so all audits run immediately without crashing.

2. Shared OS Memory Strategy (No Pinning)

How Normal Software Works: Like Chrome or MS Office, the OS manages RAM using virtual memory and memory-mapped files (mmap).

Behavior without --mlock:

- * High RAM Availability: Operating system counts memory-mapped model pages as reclaimable memory, restoring reported available RAM.
- * Dynamic Swapping: If background applications or multiple audits require memory, the OS gracefully pages idle memory to disk.
- * Crash Prevention: Instead of crashing due to memory lock failures, the system remains 100% stable. Worst-case scenario is temporary I/O latency, never a crash.

3. Recalibrated Live Resource Guardrails

Location: `src/core/resource_guard.py`

Purpose: Monitors real-time host/container memory pressure before launching audit tasks to prevent Out-Of-Memory (OOM) crashes.

Threshold Configuration:

- * WARNING Status: Triggers when available RAM falls below 15%. System logs a warning but permits audit processing.
- * CRITICAL Refusal: Triggers when available RAM falls below 8% OR absolute free RAM drops below 1.0 GB.
- * Hard Stop: Refuses NEW audit requests cleanly via HTTP 503 while allowing existing in-flight audits to finish gracefully.

4. Queue Removal & Multi-Tenancy Execution Model

Queue Removal: Removed artificial application-level semaphores in `bg_worker.py` and `port_pool.py` that forced audits to wait in line.

Immediate Triggering: When users submit audits, all audits transition to 'Running Scan' immediately.

LLM Engine Batching: llama-server natively processes multiple prompt requests using its continuous batching architecture (-np slots).

User Experience: Users never see 'Waiting in queue'. All scans process concurrently.

5. Hardware Behavior Matrix (CPU vs RAM)

CPU Utilization at 99%: Normal and safe. CPU overload causes latency/slowness, NOT system crashes.

RAM Exhaustion: High risk of OOM crash. Safeguarded by Live Resource Guard.

Machine Performance Breakdown:

- * 10 Cores / 16GB RAM: High CPU throughput. 5 parallel llama-server slots. Multiple audits run fast.
- * 4 Cores / 32GB RAM: Tons of RAM headroom (10+ audits start without hitting RAM guardrails). CPU splits 4 cores across tasks; audits complete sequentially in interleaved fashion without crashing.

6. Code Changes Summary

src/core/resource_guard.py: Recalibrated WARNING to 15%, CRITICAL to 8%, and FLOOR to 1.0 GB.

src/core/bg_worker.py: Removed _audit_semaphore queue. Audits execute immediately upon trigger.

src/core/port_pool.py: Increased max connection pool limit to 32 connections.

run_all.bat & run_all.sh: Removed --mlock flag from llama-server startup commands.